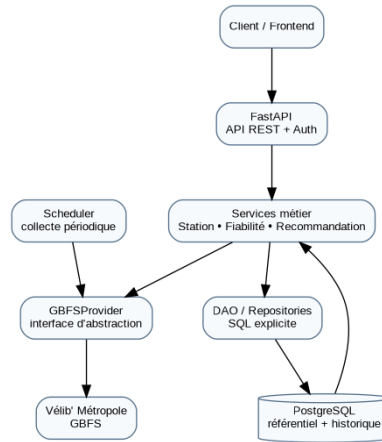


VELOSCOPE

Dossier d'analyse fonctionnelle, technique et architecturale



Projet : API d'analyse de vélos en libre-service

Technologies : Python, FastAPI, PostgreSQL, pytest, ruff, Git

API externe étudiée : Vélib' Métropole — GBFS

Version détaillée — septembre 2026

Sommaire

Partie	Contenu
1	Contexte, problématique et objectifs
2	Analyse fonctionnelle et exigences
3	Cas d'utilisation et scénarios
4	Étude de l'API GBFS / Vélib'
5	Architecture logicielle
6	Diagrammes UML et conception objet
7	Modèle de données PostgreSQL
8	Collecte, historisation et qualité des données
9	Calcul du score de fiabilité
10	Recommandation de station
11	Conception de l'API FastAPI
12	Sécurité et gestion des utilisateurs
13	Tests et qualité
14	Fonctionnalités optionnelles
15	Exploitation, performances et observabilité
16	Plan de développement
17	Risques, limites et perspectives
18	Conclusion et sources

1. Contexte, problématique et objectifs

Les systèmes de vélos en libre-service produisent une quantité importante de données temps réel : nombre de vélos, nombre de places libres, capacité des stations, disponibilité des vélos électriques et état opérationnel. Une carte classique répond à une question immédiate — « que se passe-t-il maintenant ? » — mais elle ne répond pas à des questions de fiabilité et de tendance.

VeloScope est conçu pour combler cette lacune. L'application collecte périodiquement un réseau de stations, conserve les relevés successifs et transforme cette série temporelle en informations décisionnelles. L'utilisateur peut ainsi distinguer une station simplement proche d'une station réellement fiable.

Problématique

Comment construire une API indépendante d'un fournisseur particulier capable de collecter, historiser et analyser des données GBFS afin d'estimer la fiabilité des stations et de recommander les meilleures stations à proximité ?

Objectifs techniques

- intégrer une API GBFS réelle et documentée ;
- normaliser les données externes vers un modèle interne stable ;
- historiser les états sans perte d'information ;
- fournir des requêtes temporelles performantes ;
- encapsuler les règles métier dans des services testables ;
- exposer une API REST claire avec FastAPI ;
- assurer qualité, sécurité, observabilité et maintenabilité.

Périmètre

Le MVP comprend F1 à F5. Les fonctionnalités FO1 à FO5 sont prévues comme extensions et ne doivent pas fragiliser le cœur du système. Cette séparation permet de livrer une version fonctionnelle même si toutes les fonctionnalités avancées ne sont pas réalisées.

2. Analyse fonctionnelle et exigences

Les exigences sont regroupées par acteur et par domaine. Chaque fonctionnalité doit pouvoir être associée à des endpoints, des données persistées et des tests d'acceptation.

F1 — Gestion des utilisateurs

Un utilisateur peut créer un compte, se connecter, consulter les données, ajouter ou supprimer des stations favorites. Le compte ADMIN dispose en plus de fonctions d'administration. Le système doit distinguer authentification — « qui êtes-vous ? » — et autorisation — « avez-vous le droit de faire cette action ? ».

F2 — Collecte des données

Un processus planifié interroge régulièrement les feeds GBFS. Chaque collecte produit des snapshots horodatés. Le référentiel de stations est mis à jour si les informations statiques évoluent, tandis que les snapshots sont conservés pour permettre les analyses historiques.

F3 — Consultation

Une station peut être consultée par identifiant ou recherchée géographiquement. La réponse doit présenter l'état courant, la capacité, le taux de remplissage et l'historique sélectionné. Les périodes standard sont 24 heures, 7 jours et 30 jours.

F4 — Fiabilité

La fiabilité est une métrique métier et non une propriété fournie directement par GBFS. Elle est calculée à partir de la durée passée dans les états défavorables et de la durée des épisodes. Le score doit être explicable afin qu'un utilisateur comprenne pourquoi une station est jugée fiable.

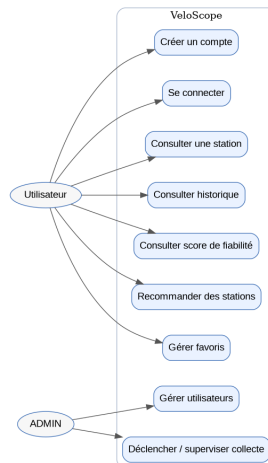
F5 — Recommandation

Le système sélectionne les stations dans un rayon donné puis les classe. La distance n'est qu'un critère : une station légèrement plus éloignée mais très fiable et bien approvisionnée peut être préférée.

Exigences non fonctionnelles

Exigence	Attente
Performance	Réponse station/recommandation généralement < 500 ms hors surcharge et réseau externe.
Disponibilité	Le système doit continuer à servir les dernières données valides si GBFS est temporairement indisponible.
Maintenabilité	Découpage par responsabilités et dépendances explicites.
Testabilité	Logique métier testable sans appeler l'API externe.
Sécurité	Hash des mots de passe, authentification, contrôle de rôle.
Qualité	ruff + pytest intégrés à la CI.

3. Cas d'utilisation et scénarios



Scénario : consulter une station

- l'utilisateur envoie l'identifiant de la station ;
- l'API vérifie que la station existe ;
- le service récupère le dernier snapshot valide ;
- le taux de remplissage est calculé ;
- l'historique demandé est récupéré si nécessaire ;
- la réponse JSON est construite à partir des DTO.

Scénario : recommander une station

- l'utilisateur fournit sa position et éventuellement le besoin e-bike ;
- le service récupère les stations dans le rayon ;
- les dernières disponibilités sont associées à chaque station ;
- les métriques historiques sont calculées ou récupérées ;
- les critères sont normalisés ;
- le score multicritère est calculé ;
- les stations sont triées et les facteurs explicatifs retournés.

Scénario : administration

Un ADMIN authentifié peut consulter les utilisateurs, modifier leur statut ou leur rôle selon les règles définies, et consulter l'état des collectes. Un utilisateur standard recevant une requête vers un endpoint ADMIN doit obtenir un HTTP 403.

4. Étude de l'API externe : GBFS et Vélib' Métropole

Le standard GBFS — General Bikeshare Feed Specification — fournit une manière normalisée de publier les informations d'un service de vélos partagés. VeloScope utilise deux catégories de données : les informations relativement stables des stations et les informations temps réel.

Vélib' Métropole

La documentation Open Data de Vélib' Métropole indique que les données sont disponibles sans clé, en JSON UTF-8, avec une mise à jour annoncée chaque minute. Les flux utiles au projet comprennent notamment gbfs.json, system_information.json, station_information.json et station_status.json. ■cite■turn0search0■

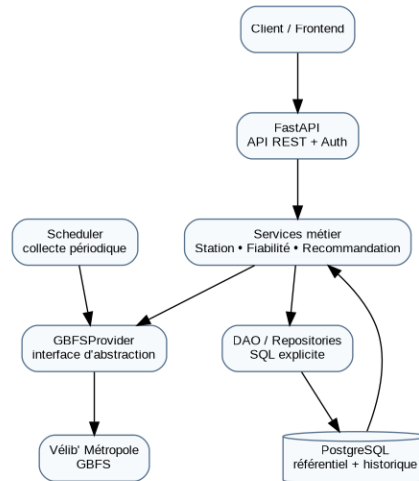
Feed	Données	Utilisation
gbfs.json	Découverte des feeds	Point d'entrée fournisseur et abstraction.
system_information.json	Identité du système, fuseau horaire	Configuration et contexte.
station_information.json	ID, nom, coordonnées, capacité	Référentiel stations.
station_status.json	Vélos, places, e-bikes, état, timestamp	Snapshots historiques.

La documentation GBFS précise que le standard décrit l'état actuel du système et n'est pas un format historique. C'est une justification centrale du projet : VeloScope doit enregistrer périodiquement les états pour construire sa propre série temporelle. ■cite■turn0search4■turn0search10■

La spécification officielle actuelle est GBFS v3.0, alors que la page Vélib' documente ses feeds sous GBFS 1.0. Il est donc recommandé de conserver une couche de mapping fournisseur/version qui convertit les champs externes vers les modèles internes. ■cite■turn0search2■turn0search0■

URL de base documentée : https://velib-metropole-opendata.smovengo.cloud/opendata/Velib_Metropole/. Cette valeur doit être placée en configuration et non codée dans les services métier. ■cite■turn0search0■

5. Architecture logicielle détaillée



Couche API

Les routes FastAPI sont responsables du protocole HTTP : parsing des paramètres, authentification, validation des DTO et conversion des exceptions métier en codes HTTP. Elles ne doivent pas contenir le calcul du score.

Couche services

Les services portent les cas d'usage : collecte, consultation, fiabilité, recommandation et gestion des favoris. Ils orchestrent le domaine, les DAO et les providers externes.

Couche DAO

Le DAO encapsule les requêtes SQL. Cette décision respecte explicitement la contrainte du sujet et facilite les tests de persistance. Les services ne construisent pas de SQL à la volée.

Couche infrastructure

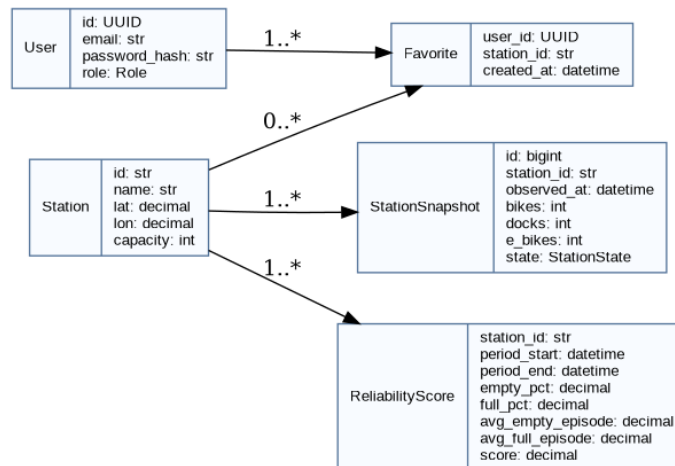
Elle contient le client HTTP GBFS, le scheduler, la configuration et les mécanismes de retry. Le client externe transforme les erreurs réseau en exceptions internes maîtrisées.

Principe de dépendance

Le sens des dépendances va vers les abstractions métier. Ainsi, AnalyticsService ne connaît ni URL Vélib', ni structure brute du JSON. Cette décision rend le système remplaçable et améliore la testabilité.

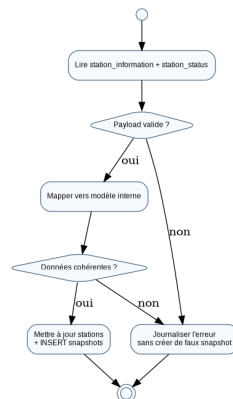
6. Conception objet et diagrammes UML

6.1 Diagramme de classes



Station représente l'identité et les informations relativement stables. StationSnapshot représente un événement d'observation. Cette distinction est fondamentale : si l'on stockait uniquement les valeurs actuelles dans Station, toute information sur le passé serait perdue.

6.2 Diagramme d'activité de collecte



La validation est effectuée avant l'insertion. Un payload invalide ne doit jamais être transformé en une série de zéros, car cela créerait artificiellement des périodes de pénurie et fausserait le score.

7. Modèle de données PostgreSQL

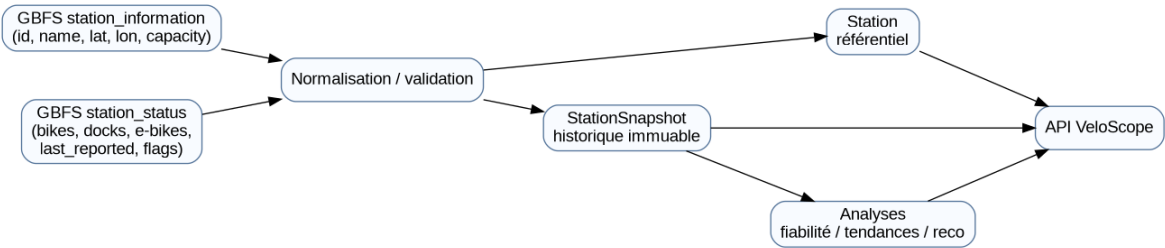


Table	Description	Choix
users	Comptes	UUID, email unique, rôle, activation.
stations	Référentiel	ID fournisseur, localisation, capacité.
station_snapshots	Historique	Une ligne par station et observation.
favorites	Favoris	Clé composite user_id/station_id.
reliability_scores	Scores calculés	Période + métriques + score.
collection_runs	Suivi des collectes	Statut, durée, erreurs, volume.

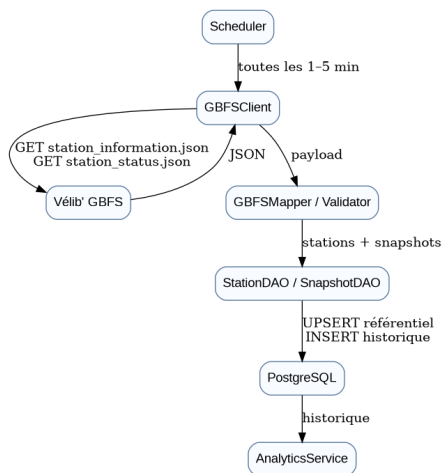
Indexation

- PK sur chaque table ;
- UNIQUE(users.email) ;
- index station_snapshots(station_id, observed_at) ;
- index reliability_scores(station_id, period_start, period_end) ;
- index géographique ou PostGIS envisageable si la recherche spatiale devient volumineuse.

Historisation

À chaque collecte, un snapshot est inséré. Une contrainte ou une logique d’idempotence peut empêcher l’insertion d’un doublon exact si le même timestamp fournisseur est reçu deux fois. Cependant, il ne faut pas remplacer un snapshot plus ancien par un plus récent avec un UPDATE.

8. Collecte, historisation et qualité des données



Une collecte peut être déclenchée par un scheduler toutes les 1 à 5 minutes. Vélib’ annonce une mise à jour chaque minute ; le standard GBFS recommande une fraîcheur adaptée au near-real-time et fixe une tolérance de cinq minutes pour certains feeds. ■cite■turn0search0■turn0search7■

Gestion temporelle

Deux timestamps sont utiles : **source_reported_at**, fourni par le flux lorsqu’il existe, et **observed_at**, correspondant au moment où VeloScope considère le relevé comme observé. Pour les analyses, il faut conserver le contexte de fraîcheur afin de ne pas confondre un état ancien avec l’état actuel.

Gestion des erreurs

Incident	Décision
Timeout	Retry limité, puis run PARTIAL/FAILED.
5xx	Retry avec backoff.
404	Erreur de configuration/feed.
JSON invalide	Rejet complet du payload.
Station incohérente	Rejet du snapshot concerné.
Feed indisponible	Pas de faux zéro ; conserver les données précédentes avec indication de fraîcheur.

Qualité métier

Des contrôles tels que $e_bikes \leq bikes$ et $bikes + docks \leq capacity$ permettent de détecter des données aberrantes. Les règles doivent toutefois tenir compte des particularités documentées du fournisseur avant de considérer toute violation comme une anomalie.

9. Calcul du score de fiabilité

La fiabilité est calculée à partir des états observés dans le temps. Une observation « vide » toutes les cinq minutes n'est pas simplement comptée comme une occurrence : elle représente une période jusqu'à l'observation suivante.

Définition des états

État	Condition métier
EMPTY	station opérationnelle et bikes = 0
FULL	station opérationnelle et docks = 0
FUNCTIONAL	station opérationnelle avec vélos et places
OFFLINE	station non opérationnelle ou données trop anciennes

Calcul des durées

Pour deux snapshots successifs t_i et t_{i+1} , l'état observé à t_i est considéré comme valable jusqu'à t_{i+1} , sous réserve d'un seuil maximal d'intervalle. Au-delà de ce seuil, l'intervalle doit être considéré comme inconnu plutôt que comme une période continue de pénurie.

Formule proposée

EmptyPenalty = empty_duration / observed_duration

FullPenalty = full_duration / observed_duration

DurationPenalty = $0.5 \times \text{norm}(\text{avg_empty_episode}) + 0.5 \times \text{norm}(\text{avg_full_episode})$

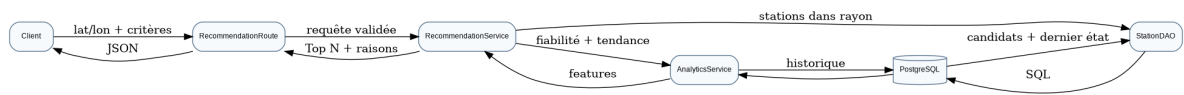
Reliability = $100 \times (1 - 0.45 \times \text{EmptyPenalty} - 0.35 \times \text{FullPenalty} - 0.20 \times \text{DurationPenalty})$

Les coefficients donnent davantage de poids au temps passé vide qu'au temps saturé, car la question principale est « vais-je trouver un vélo ? ». Ils restent néanmoins suffisamment équilibrés pour pénaliser les stations régulièrement impossibles à restituer.

Exemple

Sur 30 jours observés, une station est vide pendant 5 % du temps et saturée pendant 3 %. Ses épisodes moyens sont courts. Son score sera élevé. Une station vide 20 % du temps, même très proche, pourra être classée derrière une station située 100 mètres plus loin mais beaucoup plus fiable.

10. Moteur de recommandation



Étape 1 — sélection

Un premier filtre spatial sélectionne les stations dans le rayon demandé. Pour chaque station, la distance peut être calculée avec la formule de Haversine ou une fonction spatiale PostgreSQL/PostGIS.

Étape 2 — construction des features

Feature	Exemple
Availability	min(bikes / 10, 1)
Reliability	reliability_score / 100
EBike	e_bikes / max(1,bikes)
Trend	variation récente des vélos, normalisée
DistancePenalty	distance / rayon

Étape 3 — score

$$R = 0.35A + 0.25F + 0.15E + 0.15T - 0.10D$$

Les poids doivent être considérés comme une hypothèse de conception, pas comme une vérité universelle. Ils doivent être validés par des cas de test. Par exemple, une station à 100 m avec 0 vélo doit être moins bien classée qu'une station à 250 m avec plusieurs vélos et un historique très fiable.

Étape 4 — explicabilité

Chaque résultat renvoie les facteurs ayant contribué au classement : « 340 m », « 12 vélos », « score fiabilité 91,6 », « 5 e-bikes », « tendance stable ». L'utilisateur peut ainsi comprendre la recommandation.

11. Conception de l'API backend FastAPI

Méthode	Route	Rôle
POST	/api/v1/auth/register	Inscription
POST	/api/v1/auth/login	Authentification
GET	/api/v1/me	Profil courant
GET	/api/v1/stations	Recherche de stations
GET	/api/v1/stations/{id}	État courant
GET	/api/v1/stations/{id}/history	Historique
GET	/api/v1/stations/{id}/reliability	Fiabilité
GET	/api/v1/stations/{id}/reliability/evolution	Évolution
GET	/api/v1/recommendations	Recommandation
GET	/api/v1/favorites	Favoris
POST	/api/v1/favorites/{station_id}	Ajouter favori
DELETE	/api/v1/favorites/{station_id}	Supprimer favori
GET	/api/v1/admin/users	Administration utilisateurs
PATCH	/api/v1/admin/users/{id}	Modifier utilisateur
GET	/api/v1/admin/collection/runs	État collecte
POST	/api/v1/admin/collection/run	Collecte manuelle

Codes HTTP

Code	Signification
200	Lecture réussie
201	Ressource créée
204	Suppression réussie sans contenu
400	Paramètre ou payload invalide
401	Authentification requise
403	Droit insuffisant
404	Ressource inexistante
409	Conflit, par exemple favori déjà présent
422	Erreur de validation Pydantic
502/503	Dépendance GBFS indisponible, selon le endpoint

Les endpoints de consultation peuvent rester disponibles même si GBFS est momentanément indisponible, car ils s'appuient sur PostgreSQL. Les endpoints qui exigent l'état instantané doivent en revanche indiquer la fraîcheur de la donnée.

12. Contrats JSON et authentification

Exemple de réponse station

```
{ "id": "123", "name": "Station Exemple", "capacity": 35, "current": { "bikes": 12, "e_bikes": 5, "docks": 23, "state": "FUNCTIONAL", "observed_at": "2026-09-11T14:00:00+02:00" }, "fill_rate": 0.34, "data_age_seconds": 42 }
```

Exemple recommandation

```
{ "results": [{ "station_id": "123", "score": 0.88, "distance_m": 340, "bikes": 12, "e_bikes": 5, "reliability": 91.6, "reasons": ["proche", "fiable", "e-bike disponible"] }] }
```

Authentification

Une stratégie JWT peut être utilisée : login → access token → token transmis dans Authorization: Bearer. Le mot de passe n'est jamais stocké en clair. Le rôle ADMIN est porté par le compte et vérifié par une dépendance d'autorisation.

Une attention particulière doit être portée au compte ADMIN initial : sa création doit être contrôlée par configuration ou procédure d'administration et non par un endpoint public permettant de s'auto-attribuer le rôle.

13. Tests, qualité et intégration continue

La stratégie de tests doit vérifier les règles métier indépendamment de FastAPI et de l'API externe. Les tests d'intégration complètent ensuite cette base en vérifiant les DAO et le pipeline complet.

Type	Exemples
Domaine	EMPTY/FULL/OFFLINE ; calcul des épisodes ; score.
Mapper	Payload GBFS valide, champs manquants, version différente.
DAO	Insertion, lecture historique, agrégations, contraintes.
API	200/401/403/404/422 ; schémas JSON.
Collecte	Timeout, 5xx, JSON invalide, doublons.
Recommandation	Classement selon poids et cas limites.

Tests essentiels du score

- station toujours fonctionnelle → score proche de 100 ;
- station toujours vide → score très faible ;
- station vide 10 % du temps → pénalité correspondant à la durée ;
- trou de données long → intervalle non comptabilisé comme état ;
- station OFFLINE → ne pas compter comme EMPTY.

Qualité

ruff check . doit être exécuté avant les tests. pytest constitue le socle automatisé. Une CI Git peut exécuter lint, tests unitaires, tests d'intégration et éventuellement génération de documentation OpenAPI.

14. Fonctionnalités optionnelles

FO1 — Préviation

Une première version peut estimer $P(\text{EMPTY} \mid \text{station, jour, créneau})$ et $P(\text{FULL} \mid \text{station, jour, créneau})$. Une moyenne historique par créneau constitue une baseline. La qualité de la prévision doit être évaluée avec des métriques adaptées, par exemple précision/rappel ou Brier score pour une probabilité.

FO2 — Heatmap

La heatmap peut utiliser les coordonnées des stations et des agrégats de fiabilité. Il est préférable de calculer les agrégats côté backend plutôt que de transmettre tous les snapshots au navigateur.

FO3 — Anomalies

Les règles peuvent détecter une station sans mise à jour, des valeurs impossibles, une station figée ou des variations supérieures à la capacité. Les anomalies sont idéalement persistées dans une table dédiée pour permettre leur suivi.

FO4 — Assistant de trajet

Le système peut fournir quatre fonctions de coût : rapide, fiable, électrique et risque minimal. Un moteur de routage externe peut calculer les itinéraires, tandis que VeloScope apporte le risque associé aux stations de départ/arrivée.

FO5 — Défi utilisateurs

Cette extension nécessite de connaître les trajets réels des utilisateurs. Les seules consultations de stations ne suffisent pas à établir un trajet réellement effectué. Il faut donc un mécanisme de déclaration, d'import ou de suivi des trajets.